

Deep Learning - Homework 1

João Guilherme, ist1105939

Martin Mejer, ist1117193

Tomás Brogueira, ist1106754

December 11, 2025

Contribution Statement

João Guilherme was responsible for Question 2 (including coding and report writing). Tomás Brogueira handled Question 1, covering both code implementation and the associated report writing. Martin Mejer focused on the theoretical and mathematical aspects, completing all the derivations for Question 3 and report writing. All members reviewed the final report. We utilized Generative Artificial Intelligence (GenAI) for technical assistance and formatting tasks, specifically for creating the tables and some plots in the document, code autocompletion, and final L^AT_EX formatting.

Question 1

Part 1: Perceptron

Part A

We implemented the multi-class perceptron algorithm to classify the EMNIST handwritten letters. The model maintains a weight matrix $W \in \mathbb{R}^{26 \times 784}$. During training, we iterate through the dataset; for each misclassified example, we increase the weights corresponding to the correct class and decrease the weights corresponding to the predicted class.

The model was trained for 20 epochs. Figure 1 illustrates the training and validation accuracy over these epochs. We implemented a checkpointing system that saves the model parameters whenever the validation accuracy improves upon the previous best.

The test accuracy of the model with the best performance on the validation set is **0.6111**.

Part 2: Logistic Regression

Part A

We implemented a Logistic Regression classifier with l_2 regularization trained via Stochastic Gradient Descent (SGD). The model minimizes the cross-entropy loss between the predicted softmax probabilities and the true one-hot encoded labels.

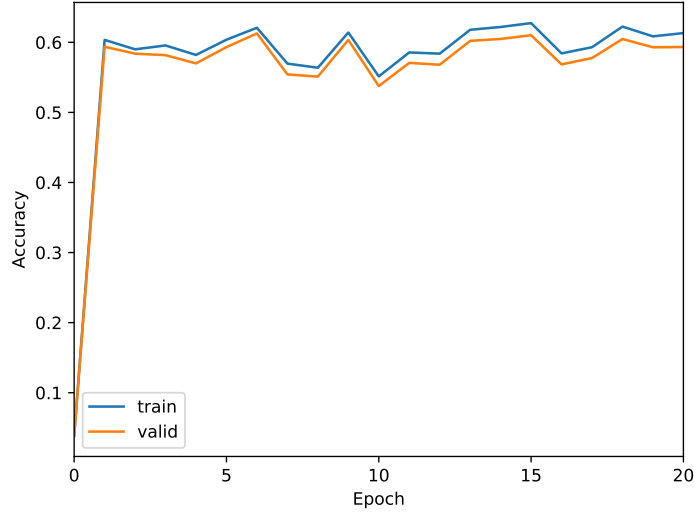


Figure 1: Training and validation accuracy over 20 epochs for the Perceptron model.

For this experiment, we used a learning rate of $\eta = 0.0001$ and an l_2 penalty of $\lambda = 0.00001$. We trained for 20 epochs. Figure 2 shows the accuracy curves.

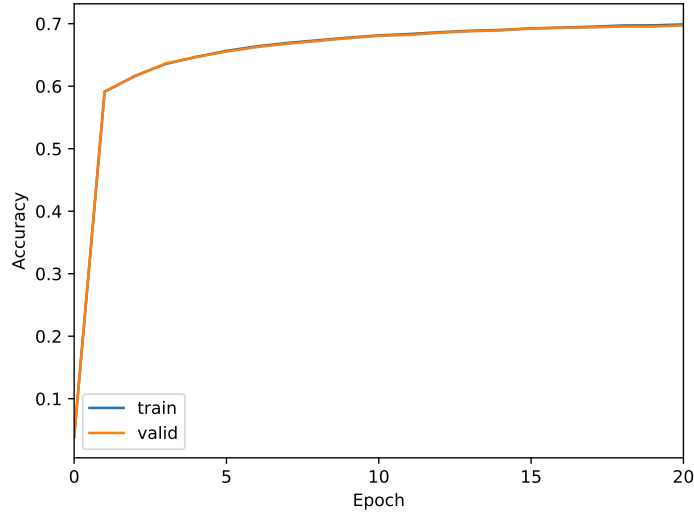


Figure 2: Training and validation accuracy over 20 epochs for the Logistic Regression model ($lr = 0.0001, l2 = 0.00001$).

The test accuracy of the best-performing checkpoint is **0.6969**.

Part B

For the alternative feature representation, we implemented **Histogram of Oriented Gradients (HOG)** [1].

Justification: HOG is a robust feature descriptor widely used in computer vision for object detection and character recognition. Unlike raw pixels, which are highly sensitive to small shifts, rotations, and illumination changes, HOG captures the local

structure and shape of the character by counting occurrences of gradient orientation in localized portions of the image.

The EMNIST letters are defined by their edges and strokes. HOG features aggregate this edge information into histograms, providing a representation that is invariant to small geometric deformations and focuses on the "shape" of the letter rather than exact pixel intensities.

Furthermore, while looking for a good representation, we tried some other well-known methods such as PCA or UMAP. However, HOG performed clearly above all other representations, equaling PCA in the time required for transformation and training, but outperforming any other representation we tried on test and validation accuracies by around 12 percentage points ($\approx 72\%$ vs $\approx 84\%$). This demonstrates that HOG is both computationally efficient and highly accurate, making it a superior representation for this task.

Part C

We performed a grid search over the following hyperparameters:

- Learning Rates: $\{0.01, 0.001, 0.0001\}$
- L2 Penalties: $\{0.001, 0.00001\}$
- Feature Representations: $\{\text{Pixel}, \text{HOG}\}$

The results are summarized in Table 1.

Table 1: Grid search results for Logistic Regression. The best configuration for each feature set is highlighted in bold.

Feature Set	Learning Rate	L2 Decay	Val Accuracy	Test Accuracy
pixel	0.0100	0.0010	0.6813	0.6792
pixel	0.0100	0.0001	0.6933	0.6972
pixel	0.0010	0.0010	0.7035	0.7052
pixel	0.0010	0.0001	0.7190	0.7217
pixel	0.0001	0.0010	0.6931	0.6921
pixel	0.0001	0.0001	0.6964	0.6964
hog	0.0100	0.0010	0.7788	0.7810
hog	0.0100	0.0001	0.8377	0.8382
hog	0.0010	0.0010	0.7803	0.7812
hog	0.0010	0.0001	0.8182	0.8213
hog	0.0001	0.0010	0.7330	0.7353
hog	0.0001	0.0001	0.7377	0.7404

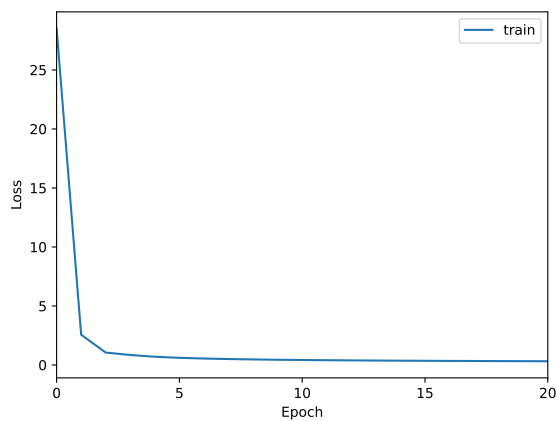
Part 3: MLP

Part A

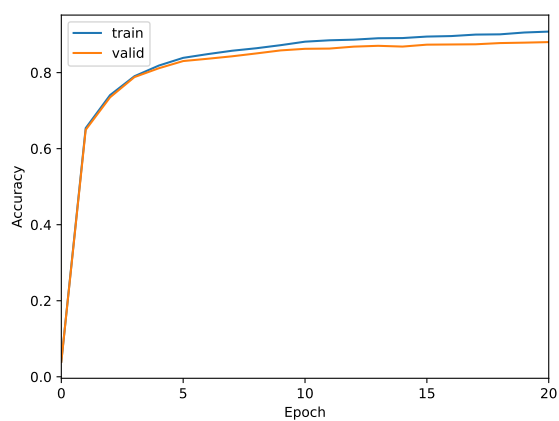
We implemented a one-hidden-layer multilayer perceptron (MLP) without using any deep learning toolkit.

The test accuracy of the best-performing checkpoint was: **0.8825**

The following are the train loss and the train and test accuracy plots requested:



(a) Training loss vs. epoch.



(b) Train and validation accuracy vs. epoch.

Figure 3: MLP training curves.

Question 2

Part 2: Towards an Infinite-Width One-Layer FFN

Part A

We decided to use the Adam optimizer with the ReLU activation. For the hyperparameters in the grid search, we considered the following values in Table 3, besides the number of hidden units.

Table 2: Hyperparameter Grid Configuration

Hyperparameter	Values to Explore
Learning Rate (η)	{0.1, 0.01, 0.001, 0.0001}
Dropout Probability (p)	{0.0, 0.5}
L2 Regularization (λ or weight decay)	{0.0, 0.01}

Table 3: Maximum validation accuracy achieved during training for different configurations of hyperparameters. The best performing configuration for each number of hidden units is highlighted in **bold**.

#	Hidden Size	Learning Rate	Dropout	L2 Decay	Val Accuracy
1	16	0.1000	0.00	0.00	0.0568
2	16	0.1000	0.00	0.01	0.2543
3	16	0.1000	0.50	0.00	0.0461
4	16	0.1000	0.50	0.01	0.1121
5	16	0.0100	0.00	0.00	0.7486
6	16	0.0100	0.00	0.01	0.6764
7	16	0.0100	0.50	0.00	0.5530
8	16	0.0100	0.50	0.01	0.5646
9	16	0.0010	0.00	0.00	0.7744
10	16	0.0010	0.00	0.01	0.7191
11	16	0.0010	0.50	0.00	0.6805
12	16	0.0010	0.50	0.01	0.6079
13	16	0.0001	0.00	0.00	0.7207
14	16	0.0001	0.00	0.01	0.6863
15	16	0.0001	0.50	0.00	0.6488
16	16	0.0001	0.50	0.01	0.6055
17	32	0.1000	0.00	0.00	0.0784
18	32	0.1000	0.00	0.01	0.2381
19	32	0.1000	0.50	0.00	0.0470
20	32	0.1000	0.50	0.01	0.1594
21	32	0.0100	0.00	0.00	0.7977
22	32	0.0100	0.00	0.01	0.6970
23	32	0.0100	0.50	0.00	0.7094

Continued on next page

Table 3 – continued from previous page

#	Hidden Size	Learning Rate	Dropout	L2 Decay	Val Accuracy
24	32	0.0100	0.50	0.01	0.6029
25	32	0.0010	0.00	0.00	0.8463
26	32	0.0010	0.00	0.01	0.7700
27	32	0.0010	0.50	0.00	0.7720
28	32	0.0010	0.50	0.01	0.6864
29	32	0.0001	0.00	0.00	0.7978
30	32	0.0001	0.00	0.01	0.7313
31	32	0.0001	0.50	0.00	0.7375
32	32	0.0001	0.50	0.01	0.6732
33	64	0.1000	0.00	0.00	0.0845
34	64	0.1000	0.00	0.01	0.2898
35	64	0.1000	0.50	0.00	0.0457
36	64	0.1000	0.50	0.01	0.1563
37	64	0.0100	0.00	0.00	0.8273
38	64	0.0100	0.00	0.01	0.7115
39	64	0.0100	0.50	0.00	0.7604
40	64	0.0100	0.50	0.01	0.6091
41	64	0.0010	0.00	0.00	0.8745
42	64	0.0010	0.00	0.01	0.7869
43	64	0.0010	0.50	0.00	0.8322
44	64	0.0010	0.50	0.01	0.7269
45	64	0.0001	0.00	0.00	0.8448
46	64	0.0001	0.00	0.01	0.7685
47	64	0.0001	0.50	0.00	0.8110
48	64	0.0001	0.50	0.01	0.7253
49	128	0.1000	0.00	0.00	0.0678
50	128	0.1000	0.00	0.01	0.2810
51	128	0.1000	0.50	0.00	0.0385
52	128	0.1000	0.50	0.01	0.1728
53	128	0.0100	0.00	0.00	0.8447
54	128	0.0100	0.00	0.01	0.7209
55	128	0.0100	0.50	0.00	0.7965
56	128	0.0100	0.50	0.01	0.6171
57	128	0.0010	0.00	0.00	0.8963
58	128	0.0010	0.00	0.01	0.7897
59	128	0.0010	0.50	0.00	0.8786
60	128	0.0010	0.50	0.01	0.7539
61	128	0.0001	0.00	0.00	0.8811
62	128	0.0001	0.00	0.01	0.7846
63	128	0.0001	0.50	0.00	0.8559
64	128	0.0001	0.50	0.01	0.7578

Continued on next page

Table 3 – continued from previous page

#	Hidden Size	Learning Rate	Dropout	L2 Decay	Val Accuracy
65	256	0.1000	0.00	0.00	0.0925
66	256	0.1000	0.00	0.01	0.2815
67	256	0.1000	0.50	0.00	0.0385
68	256	0.1000	0.50	0.01	0.1901
69	256	0.0100	0.00	0.00	0.8472
70	256	0.0100	0.00	0.01	0.7135
71	256	0.0100	0.50	0.00	0.8043
72	256	0.0100	0.50	0.01	0.6171
73	256	0.0010	0.00	0.00	0.9031
74	256	0.0010	0.00	0.01	0.7899
75	256	0.0010	0.50	0.00	0.9032
76	256	0.0010	0.50	0.01	0.7667
77	256	0.0001	0.00	0.00	0.8989
78	256	0.0001	0.00	0.01	0.7927
79	256	0.0001	0.50	0.00	0.8877
80	256	0.0001	0.50	0.01	0.7748

As can be seen in table 4, the validation accuracy monotonically increases as the layer width increases. A wider network has a higher capacity, and therefore can better approximate the objective function. If the network was overfitting to the training data, we would see a decrease in the validation accuracy for bigger layer size, which is not the case.

Table 4: Maximum Validation Accuracy Achieved by Layer Width (Best Configuration)

Hidden Size	Max Val. Accuracy	Observation
16	0.7744	Lowest accuracy.
32	0.8463	Significant increase from 16.
64	0.8745	Continued improvement.
128	0.8963	Approaching 90%.
256	0.9032	Highest overall accuracy.

Part B

The best-performing model achieved the highest validation accuracy using a learning rate of 0.001, 256 hidden units, a dropout probability of 0.5, and no L2 regularization (weight decay).

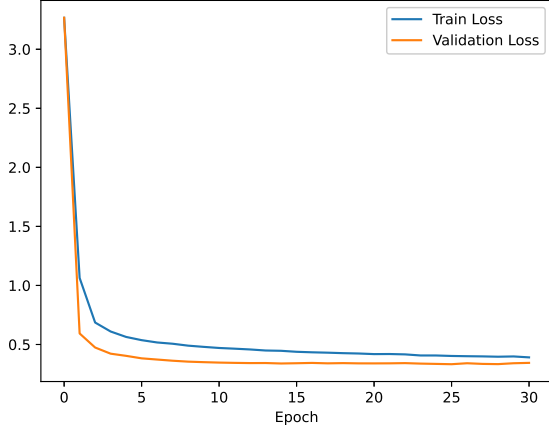


Figure 4: Training and Validation Loss over 30 Epochs. This graph shows the decrease in the cross-entropy loss function on the training and validation dataset, indicating how well the model is learning to fit the data.

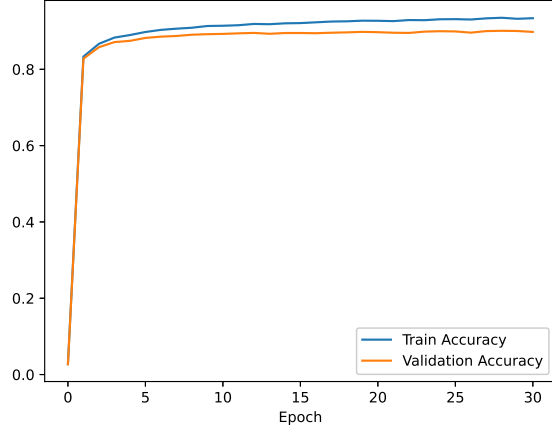


Figure 5: Training and Validation Accuracy over 30 Epochs. This graph shows the increase in the accuracy for both training and validation accuracies, which is used to monitor generalization and check for overfitting.

Based on the loss curves, the model shows no signs of underfitting, characterized by high bias. The losses have converged to a low value, which demonstrates that the network possesses sufficient complexity to minimize error and effectively capture the underlying relationships within the training data.

Regarding overfitting, the relationship between training and validation loss in Figure 4 requires careful interpretation due to the aggressive dropout applied. The validation loss remains consistently lower than the training loss because, during training, the model is forced to operate with half of its neurons randomly deactivated. This introduces significant noise and difficulty that artificially increases the training loss. However, during validation, the full network capability is restored, acting effectively as an ensemble of all the thinned networks. This results in sharper, more confident predictions and consequently a lower loss. Despite this noise during the training phase, the model achieves a high training accuracy—calculated with `model.eval()` and thus without dropout—in Figure 5 that only slightly exceeds the validation accuracy. This narrow gap indicates that the dropout successfully prevented the model from simply memorizing the training data.

This effective regularization is further confirmed by the test accuracy of 0.8976, which aligns closely with the validation performance, demonstrating that the model has achieved excellent generalization ability. We can conclude there are no signs of underfitting nor overfitting.

Part C

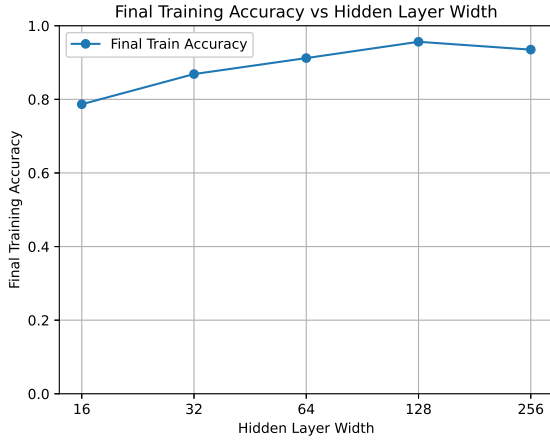


Figure 6: Final training accuracy for different network widths. We are using the best configuration for all widths. Layer width 256 has a **dropout of 0.50**, while the remaining layers have no dropout.

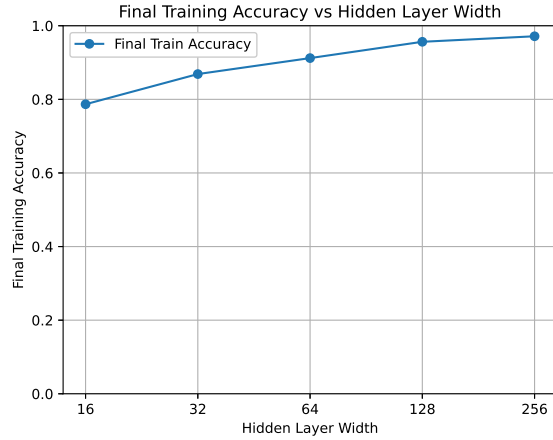


Figure 7: Final training accuracy for different network widths. We are **not** using the best configuration for the width 256. We kept the same configuration for width 256 but use **no dropout**.

From Figure 6, the training accuracy initially increases monotonically up to a width of 128 units, after which it drops for the width of 256. This result must be looked at carefully because, while all other widths used a dropout probability of 0%, the 256-unit layer size used a dropout probability of 50%. This means that during training, hidden units are randomly removed. This has the effect of lowering the training accuracy because the model is forced to learn a less expressive function at each training step, and only a fraction of the total possible connections are active for computing the loss. Therefore, up to a width of 128 units, we can experimentally verify the Universal Approximation Theorem [3], where the model approaches a train accuracy of 1.0, effectively approximating arbitrarily well any continuous function, given enough hidden layers. For an infinite-width one-layer FFN, we would expect a training accuracy of 1.0, provided, of course, no dropout is used. We also plotted without using any dropout (Fig. 7), and we can see clearly the influence that dropout had, confirming that the Universal Approximation Theorem holds.

Part 3: Effect of Depth in Vanilla FFNs

Part A

Table 5: Best Validation Accuracy as a Function of Network Depth

Number of Hidden Layers	Best Validation Accuracy
1	0.8431
3	0.8689
5	0.8719
7	0.8581
9	0.8526

The validation accuracy increases up to 5 hidden layers, peaking at 0.8719, and then begins to decrease as the network depth further increases to 7 and 9 layers. Since the number of linear regions carved out by a deep neural network with fixed layer width increases exponentially with depth [5], deeper models in principle have the capacity to represent more complex decision boundaries. However, this additional expressive power does not necessarily translate into better generalization. Beyond a certain depth, the optimization landscape becomes increasingly difficult to navigate, leading to issues such as vanishing gradients, suboptimal local minima, or overfitting to the training data.

Part B

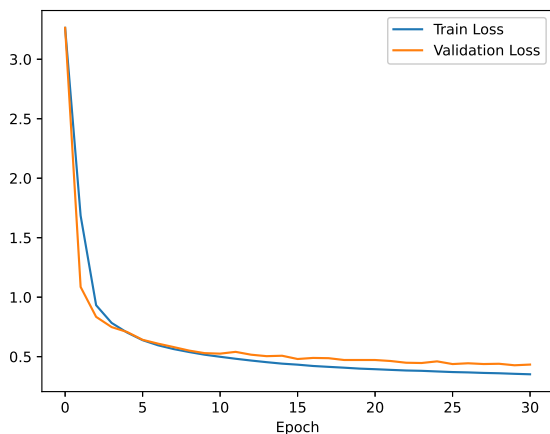


Figure 8: Loss Curves for Depth 5, Width 32. This graph shows the decrease in the cross-entropy loss function over epochs for both the training and validation datasets.

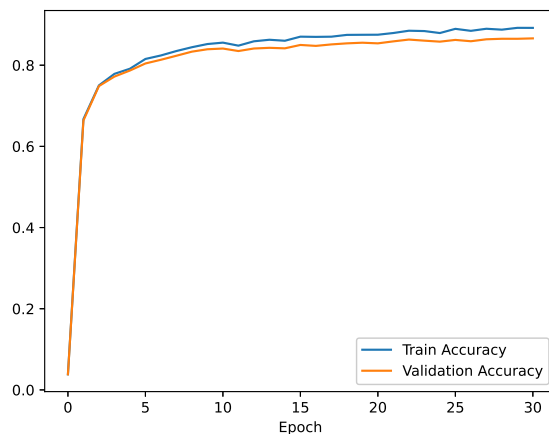


Figure 9: Accuracy Curves for Depth 5, Width 32. This graph displays the classification accuracy over epochs for both the training and validation datasets.

The model demonstrates clear convergence. The training loss curve shows a steep initial decline before stabilizing at a low value, a trend that is mirrored by the validation loss which also settles at a minimal point, converging closely together. The training accuracy and validation accuracy increase rapidly at the beginning but then converge slowly to a value close to 1. There is no indication of overfit or underfit. The model

achieves very low training and validation losses alongside high training and validation accuracies, indicating it has sufficient capacity to capture the underlying patterns in the data (low bias). The validation accuracy remains non-decreasing and follows the training accuracy trend closely. There is no divergence where training loss continues to drop while validation loss increases or validation accuracy degrades (which would indicate high variance). The model achieved a final test accuracy of 0.8681. The test accuracy aligns closely with the training accuracy, confirming that the model performs reliably on unseen data and is not just memorizing the training set.

Part C

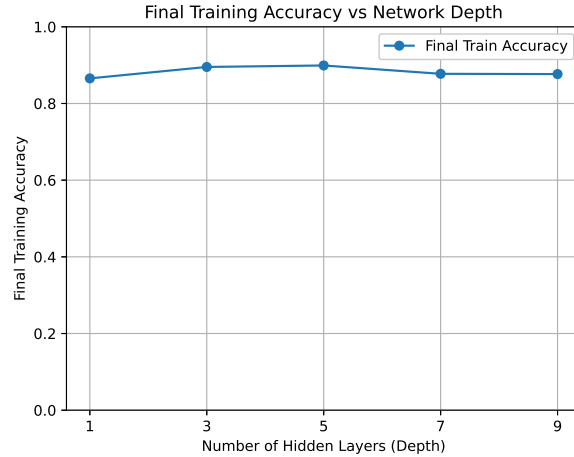


Figure 10: Caption

For depths up to 5, the increasing training accuracy demonstrates that adding layers expands the model’s representational capacity [5]. However, the subsequent drop in accuracy for depths 7 and 9 reveals that increased capacity does not guarantee better performance due to optimization challenges. As the network deepens, the optimization landscape becomes increasingly non-convex and chaotic, making it difficult for standard gradient descent to locate the global minimum.

Furthermore, these deeper architectures likely suffer from the vanishing gradient problem, where gradients diminish exponentially as they propagate backward through the layers. This phenomenon prevents weights in the earlier layers from updating effectively, effectively stalling the learning process. Consequently, despite possessing the theoretical capacity to perfectly interpolate the training data, the deeper models fail to converge to an optimal solution, causing the observed degradation in training accuracy.

To prevent the issue of vanishing gradients, one could use Deep Residual Learning [2], where “skip connections” are introduced to bypass one or more layers.

Question 3.1: ReLU as Projection

Proposition 1. For any vector $\mathbf{z} \in \mathbb{R}^K$, we want to show

$$\text{relu}(\mathbf{z}) = \arg \min_{\mathbf{y} \geq 0} \|\mathbf{y} - \mathbf{z}\|^2,$$

where the inequality $\mathbf{y} \geq 0$ is applied element-wise.

Proof. We will define $\mathbf{y}^*$ to be the solution to the optimization problem:

$$\mathbf{y}^* = \arg \min_{\mathbf{y} \geq 0} \|\mathbf{y} - \mathbf{z}\|^2.$$

We need to show that $\text{relu}(\mathbf{z})$ is the solution to the constrained optimization problem of projecting $\mathbf{z}$ onto the non-negative orthant $\mathbb{R}_+^K = \{\mathbf{y} \in \mathbb{R}^K : y_i \geq 0 \text{ for all } i\}$.

The objective function is

$$f(\mathbf{y}) = \|\mathbf{y} - \mathbf{z}\|^2 = \sum_{i=1}^K (y_i - z_i)^2.$$

Since the objective decomposes as a sum over independent coordinates, we can minimize each coordinate separately:

$$\min_{\mathbf{y} \geq 0} \|\mathbf{y} - \mathbf{z}\|^2 = \sum_{i=1}^K \min_{y_i \geq 0} (y_i - z_i)^2.$$

For each coordinate i , we solve

$$\min_{y_i \geq 0} (y_i - z_i)^2.$$

Taking the derivative with respect to y_i gives

$$\frac{d}{dy_i} (y_i - z_i)^2 = 2(y_i - z_i).$$

Case 1: If $z_i > 0$, the unconstrained minimum occurs at $y_i = z_i$, which satisfies the constraint $y_i \geq 0$. Hence $y_i^* = z_i$.

Case 2: If $z_i \leq 0$, the unconstrained minimum $y_i = z_i$ violates $y_i \geq 0$. Because $(y_i - z_i)^2$ is convex and grows as y_i moves away from z_i , the constrained minimum occurs at the boundary: $y_i^* = 0$.

Combining both cases,

$$y_i^* = \begin{cases} z_i & \text{if } z_i > 0, \\ 0 & \text{if } z_i \leq 0, \end{cases} = \max(0, z_i) = \text{relu}(z_i).$$

Since this holds for every i , it holds for the whole vector:

$$\mathbf{y}^* = \text{relu}(\mathbf{z}) = \arg \min_{\mathbf{y} \geq 0} \|\mathbf{y} - \mathbf{z}\|^2.$$

This is precisely what we wanted to show. □

Question 3.2: Properties of Activation Functions

Part 1: Constant Insensitivity

Proposition 2. *For any vector $\mathbf{z} \in \mathbb{R}^K$ and any constant $c \in \mathbb{R}$, the softmax, sparsemax, and relumax activations are all insensitive to adding a constant to $\mathbf{z}$. That is:*

$$\begin{aligned}\text{softmax}(\mathbf{z} + c\mathbf{1}) &= \text{softmax}(\mathbf{z}) \\ \text{sparsemax}(\mathbf{z} + c\mathbf{1}) &= \text{sparsemax}(\mathbf{z}) \\ \text{relumax}_b(\mathbf{z} + c\mathbf{1}) &= \text{relumax}_b(\mathbf{z})\end{aligned}$$

where $\mathbf{1} = [1, 1, \dots, 1]^T$ is the all-ones vector.

Proof for Softmax

Proof. Recall that softmax is defined as:

$$\text{softmax}(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

For $\mathbf{z}' = \mathbf{z} + c\mathbf{1}$, we have $z'_i = z_i + c$ for all i . Then:

$$\begin{aligned}\text{softmax}(\mathbf{z}')_i &= \frac{e^{z'_i}}{\sum_{j=1}^K e^{z'_j}} \\ &= \frac{e^{z_i + c}}{\sum_{j=1}^K e^{z_j + c}} \\ &= \frac{e^{z_i} \cdot e^c}{\sum_{j=1}^K (e^{z_j} \cdot e^c)} \\ &= \frac{e^{z_i} \cdot e^c}{e^c \sum_{j=1}^K e^{z_j}} \\ &= \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} \\ &= \text{softmax}(\mathbf{z})_i\end{aligned}$$

Therefore, $\text{softmax}(\mathbf{z} + c\mathbf{1}) = \text{softmax}(\mathbf{z})$. □

Proof for Sparsemax

Proof. Recall that sparsemax is defined as:

$$\text{sparsemax}(\mathbf{z}) = \arg \min_{\mathbf{p} \in \Delta_K} \|\mathbf{p} - \mathbf{z}\|^2$$

where $\Delta_K = \{\mathbf{y} \in \mathbb{R}^K : \mathbf{y} \geq 0, \sum_i y_i = 1\}$ is the probability simplex.

For $\mathbf{z}' = \mathbf{z} + c\mathbf{1}$, consider the optimization problem:

$$\text{sparsemax}(\mathbf{z}') = \arg \min_{\mathbf{p} \in \Delta_K} \|\mathbf{p} - \mathbf{z}'\|^2 = \arg \min_{\mathbf{p} \in \Delta_K} \|\mathbf{p} - (\mathbf{z} + c\mathbf{1})\|^2$$

Expanding the objective:

$$\begin{aligned}
\|\mathbf{p} - (\mathbf{z} + c\mathbf{1})\|^2 &= \sum_{i=1}^K (p_i - z_i - c)^2 \\
&= \sum_{i=1}^K [(p_i - z_i) - c]^2 \\
&= \sum_{i=1}^K [(p_i - z_i)^2 - 2c(p_i - z_i) + c^2] \\
&= \|\mathbf{p} - \mathbf{z}\|^2 - 2c \sum_{i=1}^K (p_i - z_i) + Kc^2
\end{aligned}$$

Since $p \in \Delta_K$, we have $\sum_{i=1}^K p_i = 1$. Therefore:

$$\begin{aligned}
\sum_{i=1}^K (p_i - z_i) &= \sum_{i=1}^K p_i - \sum_{i=1}^K z_i \\
&= 1 - \sum_{i=1}^K z_i
\end{aligned}$$

This quantity is independent of p , so:

$$\|\mathbf{p} - (\mathbf{z} + c\mathbf{1})\|^2 = \|\mathbf{p} - \mathbf{z}\|^2 - 2c \left(1 - \sum_{i=1}^K z_i\right) + Kc^2$$

The terms $-2c(1 - \sum_{i=1}^K z_i)$ and Kc^2 are constant with respect to p . Therefore:

$$\arg \min_{p \in \Delta_K} \|p - (\mathbf{z} + c\mathbf{1})\|^2 = \arg \min_{p \in \Delta_K} \|p - \mathbf{z}\|^2$$

Thus, $\text{sparsemax}(\mathbf{z} + c\mathbf{1}) = \text{sparsemax}(\mathbf{z})$. □

Proof for Relumax

Proof. Recall that relumax is defined as:

$$\text{relumax}_b(\mathbf{z})_i = \frac{\text{relu}(z_i - \max(\mathbf{z}) + b)}{\sum_{j=1}^K \text{relu}(z_j - \max(\mathbf{z}) + b)}$$

For $\mathbf{z}' = \mathbf{z} + c\mathbf{1}$, we have $z'_i = z_i + c$ for all i . First, note that:

$$\max(\mathbf{z}') = \max_i (z_i + c) = \max(\mathbf{z}) + c$$

Now compute the numerator:

$$\begin{aligned}
\text{relu}(z'_i - \max(\mathbf{z}') + b) &= \text{relu}((z_i + c) - (\max(\mathbf{z}) + c) + b) \\
&= \text{relu}(z_i - \max(\mathbf{z}) + b)
\end{aligned}$$

Similarly, for the denominator:

$$\begin{aligned}\sum_{j=1}^K \text{relu}(z'_j - \max(z') + b) &= \sum_{j=1}^K \text{relu}((z_j + c) - (\max(z) + c) + b) \\ &= \sum_{j=1}^K \text{relu}(z_j - \max(z) + b)\end{aligned}$$

Therefore:

$$\text{relumax}_b(z')_i = \frac{\text{relu}(z_i - \max(z) + b)}{\sum_{j=1}^K \text{relu}(z_j - \max(z) + b)} = \text{relumax}_b(z)_i$$

Thus, $\text{relumax}_b(z + c\mathbf{1}) = \text{relumax}_b(z)$. $\square$

Part 2: Zero Temperature Limit

Proposition 3. *Assuming all entries in z are distinct, softmax, sparsemax, and relumax all approach a one-hot vector in the zero temperature limit:*

$$\lim_{T \rightarrow 0^+} \text{softmax}(z/T) = \lim_{T \rightarrow 0^+} \text{sparsemax}(z/T) = \lim_{T \rightarrow 0^+} \text{relumax}_b(z/T) = e_k$$

where $k = \arg \max_j z_j$ and e_k is the k -th standard basis vector (one-hot vector with 1 in position k and 0 elsewhere).

Proof for Softmax

Proof. Let $k = \arg \max_j z_j$. Without loss of generality, assume $z_k = \max(z)$. We have:

$$\text{softmax}(z/T)_i = \frac{e^{z_i/T}}{\sum_{j=1}^K e^{z_j/T}}$$

Dividing numerator and denominator by $e^{z_k/T}$:

$$\text{softmax}(z/T)_i = \frac{e^{(z_i - z_k)/T}}{\sum_{j=1}^K e^{(z_j - z_k)/T}}$$

For $i = k$:

$$\text{softmax}(z/T)_k = \frac{e^0}{\sum_{j=1}^K e^{(z_j - z_k)/T}} = \frac{1}{1 + \sum_{j \neq k} e^{(z_j - z_k)/T}}$$

Since $z_j < z_k$ for all $j \neq k$ (all entries distinct), we have $z_j - z_k < 0$. Therefore, as $T \rightarrow 0^+$:

$$e^{(z_j - z_k)/T} \rightarrow 0 \quad \text{for all } j \neq k$$

Thus:

$$\lim_{T \rightarrow 0^+} \text{softmax}(z/T)_k = \frac{1}{1 + 0} = 1$$

For $i \neq k$:

$$\lim_{T \rightarrow 0^+} \text{softmax}(z/T)_i = \lim_{T \rightarrow 0^+} \frac{e^{(z_i - z_k)/T}}{1 + \sum_{j \neq k} e^{(z_j - z_k)/T}} = \frac{0}{1 + 0} = 0$$

Therefore, $\lim_{T \rightarrow 0^+} \text{softmax}(z/T) = e_k$. $\square$

Proof for Sparsemax

Proof. Recall that $\text{sparsemax}(z)_i = \text{relu}(z_i - \tau)$ where τ satisfies $\sum_i \text{relu}(z_i - \tau) = 1$.

For the scaled input z/T , we have:

$$\text{sparsemax}(z/T)_i = \text{relu}\left(\frac{z_i}{T} - \tau_T\right)$$

where τ_T satisfies $\sum_i \text{relu}\left(\frac{z_i}{T} - \tau_T\right) = 1$.

Let $k = \arg \max_j z_j$. As $T \rightarrow 0^+$, the values z_i/T become increasingly separated. Specifically, $z_k/T \rightarrow +\infty$ while $(z_k - z_j)/T \rightarrow +\infty$ for all $j \neq k$.

To maintain the constraint $\sum_i \text{relu}(z_i/T - \tau_T) = 1$, we must have $\tau_T \rightarrow +\infty$ and specifically $\tau_T \approx z_k/T$ (but slightly less).

For $i = k$: Since $z_k/T - \tau_T$ must be positive and the sum equals 1, we have:

$$\text{relu}(z_k/T - \tau_T) \rightarrow 1$$

For $i \neq k$: Since $z_i < z_k$, we have $z_i/T < z_k/T$. As $\tau_T \approx z_k/T$, we get $z_i/T - \tau_T < 0$, so:

$$\text{relu}(z_i/T - \tau_T) = 0$$

Therefore, $\lim_{T \rightarrow 0^+} \text{sparsemax}(z/T) = e_k$. □

Proof for Relumax

Proof. We have:

$$\text{relumax}_b(z/T)_i = \frac{\text{relu}(z_i/T - \max(z)/T + b)}{\sum_{j=1}^K \text{relu}(z_j/T - \max(z)/T + b)}$$

Let $k = \arg \max_j z_j$, so $\max(z) = z_k$. Then:

$$\text{relumax}_b(z/T)_i = \frac{\text{relu}((z_i - z_k)/T + b)}{\sum_{j=1}^K \text{relu}((z_j - z_k)/T + b)}$$

For $i = k$:

$$\text{relumax}_b(z/T)_k = \frac{\text{relu}(b)}{\text{relu}(b) + \sum_{j \neq k} \text{relu}((z_j - z_k)/T + b)}$$

For $i \neq k$: Since $z_i < z_k$ (all entries distinct), we have $(z_i - z_k)/T < 0$ and as $T \rightarrow 0^+$:

$$\frac{z_i - z_k}{T} \rightarrow -\infty$$

Therefore, for any fixed $b > 0$:

$$\frac{z_i - z_k}{T} + b \rightarrow -\infty \quad \text{as } T \rightarrow 0^+$$

This means:

$$\text{relu}\left(\frac{z_i - z_k}{T} + b\right) \rightarrow 0 \quad \text{for all } i \neq k$$

Thus:

$$\lim_{T \rightarrow 0^+} \text{relumax}_b(z/T)_k = \frac{\text{relu}(b)}{\text{relu}(b) + 0} = 1$$

and for $i \neq k$:

$$\lim_{T \rightarrow 0^+} \text{relumax}_b(z/T)_i = 0$$

Therefore, $\lim_{T \rightarrow 0^+} \text{relumax}_b(z/T) = e_k$. $\square$

Part 3: Equivalence between Relumax and Sparsemax

Proposition 4. *For each $z \in \mathbb{R}^K$, there exists a value $b > 0$ such that $\text{relumax}_b(z) = \text{sparsemax}(z)$.*

Proof. Recall that $\text{sparsemax}(z)_i = \text{relu}(z_i - \tau)$ where τ is the unique scalar satisfying $\sum_i \text{relu}(z_i - \tau) = 1$.

We have:

$$\text{relumax}_b(z)_i = \frac{\text{relu}(z_i - \max(z) + b)}{\sum_{j=1}^K \text{relu}(z_j - \max(z) + b)}$$

Let τ be the threshold for sparsemax. Define:

$$b^* = \tau - \max(z)$$

Note that for sparsemax to produce a valid probability distribution with at least one non-zero entry, we must have $\tau < \max(z)$ (otherwise the maximum entry would be mapped to 0 or negative). However, we need $b > 0$ for relumax.

Actually, let's reconsider. The support of sparsemax (indices where the output is non-zero) is given by $S = \{i : z_i > \tau\}$. For indices in S :

$$\text{sparsemax}(z)_i = z_i - \tau > 0$$

Now, for relumax with parameter b , the support is $S_b = \{i : z_i > \max(z) - b\}$.

For the two to be equal, we need: 1. The same support: $S = S_b$, which requires $\tau = \max(z) - b$, or equivalently $b = \max(z) - \tau$. 2. The same values on the support.

From condition 1, we need $b = \max(z) - \tau$. Since $\tau < \max(z)$ (the maximum entry must be in the support), we have $b > 0$.

With this choice of b :

$$\text{relu}(z_i - \max(z) + b) = \text{relu}(z_i - \max(z) + (\max(z) - \tau)) = \text{relu}(z_i - \tau)$$

Therefore:

$$\text{relumax}_b(z)_i = \frac{\text{relu}(z_i - \tau)}{\sum_{j=1}^K \text{relu}(z_j - \tau)}$$

Now, sparsemax satisfies the constraint $\sum_i \text{sparsemax}(z)_i = \sum_i \text{relu}(z_i - \tau) = 1$. Thus:

$$\text{relumax}_b(z)_i = \frac{\text{relu}(z_i - \tau)}{1} = \text{relu}(z_i - \tau) = \text{sparsemax}(z)_i$$

Therefore, with $b = \max(z) - \tau > 0$, we have $\text{relumax}_b(z) = \text{sparsemax}(z)$. $\square$

Question 3.3: Closed Form Expressions for $K = 2$

For this problem, we consider $K = 2$ and $z = [0, t]$.

Softmax

Expression for $\text{softmax}(z)_2$

$$\begin{aligned}\text{softmax}(z)_2 &= \frac{e^{z_2}}{e^{z_1} + e^{z_2}} \\ &= \frac{e^t}{e^0 + e^t} \\ &= \frac{e^t}{1 + e^t} \\ &= \frac{1}{1 + e^{-t}}\end{aligned}$$

This is the standard logistic sigmoid function: $\sigma(t) = \frac{1}{1+e^{-t}}$.

Derivative with respect to t

$$\begin{aligned}\frac{d}{dt}\text{softmax}(z)_2 &= \frac{d}{dt} \left(\frac{1}{1 + e^{-t}} \right) \\ &= -\frac{1}{(1 + e^{-t})^2} \cdot \frac{d}{dt}(1 + e^{-t}) \\ &= -\frac{1}{(1 + e^{-t})^2} \cdot (-e^{-t}) \\ &= \frac{e^{-t}}{(1 + e^{-t})^2} \\ &= \frac{1}{1 + e^{-t}} \cdot \frac{e^{-t}}{1 + e^{-t}} \\ &= \frac{1}{1 + e^{-t}} \cdot \left(1 - \frac{1}{1 + e^{-t}} \right) \\ &= \sigma(t)(1 - \sigma(t))\end{aligned}$$

Sparsemax

Expression for $\text{sparsemax}(z)_2$

Recall that $\text{sparsemax}(z)_i = \text{relu}(z_i - \tau)$ where τ satisfies $\sum_i \text{relu}(z_i - \tau) = 1$.

For $z = [0, t]$, we need to find τ such that:

$$\text{relu}(0 - \tau) + \text{relu}(t - \tau) = 1$$

We define p_2 to be the second coordinate: $p_2 = \text{relu}(t - \tau)$. We consider different cases

based on the value of t :

Case 1: $t \leq -1$

In this region, t is significantly smaller than 0. We hypothesize that only the first component ($z_1 = 0$) is active.

$$\text{relu}(0 - \tau) = 1 \implies -\tau = 1 \implies \tau = -1$$

Validity check: For the second component to be inactive, we require $t - \tau \leq 0$. Substituting $\tau = -1$, we get $t - (-1) \leq 0 \implies t \leq -1$, which holds.

Thus, $p_2 = \text{relu}(t - (-1)) = 0$.

Case 2: $-1 < t < 1$

Here, t is close to 0. We hypothesize that both components are active.

$$(0 - \tau) + (t - \tau) = 1 \implies t - 2\tau = 1 \implies \tau = \frac{t - 1}{2}$$

Validity check: We require both arguments to be positive:

- $0 - \tau > 0 \implies \tau < 0 \implies \frac{t-1}{2} < 0 \implies t < 1$.
- $t - \tau > 0 \implies \tau < t \implies \frac{t-1}{2} < t \implies t > -1$.

Both conditions hold in this interval.

Thus, $p_2 = t - \frac{t-1}{2} = \frac{t+1}{2}$.

Case 3: $t \geq 1$

In this region, t is significantly larger than 0. We hypothesize that only the second component ($z_2 = t$) is active.

$$\text{relu}(t - \tau) = 1 \implies t - \tau = 1 \implies \tau = t - 1$$

Validity check: For the first component to be inactive, we require $0 - \tau \leq 0$. Substituting $\tau = t - 1$, we get $-(t - 1) \leq 0 \implies t \geq 1$, which holds.

Thus, $p_2 = \text{relu}(t - (t - 1)) = 1$.

Summary

The second coordinate of the sparsemax output is the same as a shifted and stretched ReLU function clipped to be maximum 1 (aka. a Hard Sigmoid function):

$$\text{sparsemax}(z)_2 = \begin{cases} 0 & \text{if } t \leq -1 \\ \frac{t+1}{2} & \text{if } -1 < t < 1 \\ 1 & \text{if } t \geq 1 \end{cases}$$

Derivative with respect to t

The derivative is defined everywhere except at $t = -1$ and $t = 1$:

$$\frac{d}{dt} \text{sparsemax}(z)_2 = \begin{cases} 0 & \text{if } t < -1 \\ \frac{1}{2} & \text{if } -1 < t < 1 \\ 0 & \text{if } t > 1 \end{cases}$$

Relumax

Expression for $\text{relumax}_b(z)_2$

For $z = [0, t]$, we have $\max(z) = \max(0, t)$.

Case 1: $t \leq 0$

Then $\max(z) = 0$, and:

$$\begin{aligned}\text{relumax}_b(z)_2 &= \frac{\text{relu}(t - 0 + b)}{\text{relu}(0 - 0 + b) + \text{relu}(t - 0 + b)} \\ &= \frac{\text{relu}(t + b)}{\text{relu}(b) + \text{relu}(t + b)} \\ &= \frac{\text{relu}(t + b)}{b + \text{relu}(t + b)}\end{aligned}$$

If $t + b > 0$ (i.e., $t > -b$):

$$\text{relumax}_b(z)_2 = \frac{t + b}{b + (t + b)} = \frac{t + b}{t + 2b}$$

If $t + b \leq 0$ (i.e., $t \leq -b$):

$$\text{relumax}_b(z)_2 = \frac{0}{b + 0} = 0$$

Case 2: $t > 0$

Then $\max(z) = t$, and:

$$\begin{aligned}\text{relumax}_b(z)_2 &= \frac{\text{relu}(t - t + b)}{\text{relu}(0 - t + b) + \text{relu}(t - t + b)} \\ &= \frac{\text{relu}(b)}{\text{relu}(b - t) + \text{relu}(b)} \\ &= \frac{b}{\text{relu}(b - t) + b}\end{aligned}$$

If $b - t > 0$ (i.e., $t < b$):

$$\text{relumax}_b(z)_2 = \frac{b}{(b - t) + b} = \frac{b}{2b - t}$$

If $b - t \leq 0$ (i.e., $t \geq b$):

$$\text{relumax}_b(z)_2 = \frac{b}{0 + b} = 1$$

Summary:

$$\text{relumax}_b(z)_2 = \begin{cases} 0 & \text{if } t \leq -b \\ \frac{t+b}{t+2b} & \text{if } -b < t \leq 0 \\ \frac{b}{2b-t} & \text{if } 0 < t < b \\ 1 & \text{if } t \geq b \end{cases}$$

Derivative with respect to t

For $t \in (-b, 0)$:

$$\frac{d}{dt} \left(\frac{t+b}{t+2b} \right) = \frac{(t+2b) - (t+b)}{(t+2b)^2} = \frac{b}{(t+2b)^2}$$

For $t \in (0, b)$:

$$\frac{d}{dt} \left(\frac{b}{2b-t} \right) = b \cdot \frac{-1 \cdot (-1)}{(2b-t)^2} = \frac{b}{(2b-t)^2}$$

Combined:

$$\frac{d}{dt} \text{relumax}_b(z)_2 = \begin{cases} 0 & \text{if } t < -b \\ \text{undefined} & \text{if } t = -b \\ \frac{b}{(t+2b)^2} & \text{if } -b < t < 0 \\ \text{undefined} & \text{if } t = 0 \\ \frac{b}{(2b-t)^2} & \text{if } 0 < t < b \\ \text{undefined} & \text{if } t = b \\ 0 & \text{if } t > b \end{cases}$$

Question 3.4: Jacobian of relumax_b

We are given, for $z \in \mathbb{R}^K$,

$$\text{relumax}_b(z)_i = \frac{\text{relu}(z_i - \max(z) + b)}{\sum_{k=1}^K \text{relu}(z_k - \max(z) + b)}.$$

We want to compute, for each i, j ,

$$\frac{\partial \text{relumax}_b(z)_i}{\partial z_j}.$$

We will assume we are at a point z where

- there is a unique index t such that $z_t = \max(z)$, and
- for all k , $z_k - \max(z) + b \neq 0$
- the denominator is nonzero, i.e. $\sum_{k=1}^K \text{relu}(z_k - \max(z) + b) > 0$ (at least one ReLU term is active),

so that both $\max(\cdot)$ and $\text{relu}(\cdot)$ are differentiable in the required directions.

Define

$$y_i(z) = \text{relumax}_b(z)_i = \frac{\text{relu}(z_i - \max(z) + b)}{\sum_{k=1}^K \text{relu}(z_k - \max(z) + b)}.$$

Step 1: Quotient rule

Write

$$\text{numerator}_i(z) = \text{relu}(z_i - \max(z) + b), \quad \text{denominator}(z) = \sum_{k=1}^K \text{relu}(z_k - \max(z) + b).$$

Then

$$y_i(z) = \frac{\text{numerator}_i(z)}{\text{denominator}(z)},$$

so by the quotient rule

$$\frac{\partial y_i}{\partial z_j} = \frac{\frac{\partial \text{numerator}_i}{\partial z_j} \text{denominator} - \text{numerator}_i \frac{\partial \text{denominator}}{\partial z_j}}{\text{denominator}^2}. \quad (1)$$

All the z -dependence is implicit to keep notation light.

Step 2: Derivative of the numerator

Recall that for $a \neq 0$,

$$\frac{d}{da} \text{relu}(a) = \begin{cases} 1, & a > 0, \\ 0, & a < 0. \end{cases}$$

Now

$$\text{numerator}_i(z) = \text{relu}(z_i - \max(z) + b).$$

Its derivative w.r.t. z_j is

$$\frac{\partial \text{numerator}_i}{\partial z_j} = \begin{cases} \frac{\partial}{\partial z_j} (z_i - \max(z) + b), & \text{if } z_i - \max(z) + b > 0, \\ 0, & \text{if } z_i - \max(z) + b < 0. \end{cases}$$

We have

$$\frac{\partial}{\partial z_j} (z_i - \max(z) + b) = \frac{\partial z_i}{\partial z_j} - \frac{\partial \max(z)}{\partial z_j} = \delta_{ij} - \frac{\partial \max(z)}{\partial z_j},$$

where δ_{ij} is the Kronecker delta.

Since there is a unique index t with $z_t = \max(z)$, the derivative of $\max(z)$ is

$$\frac{\partial \max(z)}{\partial z_j} = \begin{cases} 1, & j = t, \\ 0, & j \neq t, \end{cases} = \delta_{tj}.$$

Therefore

$$\frac{\partial \text{numerator}_i}{\partial z_j} = \begin{cases} \delta_{ij} - \delta_{tj}, & \text{if } z_i - \max(z) + b > 0, \\ 0, & \text{if } z_i - \max(z) + b < 0. \end{cases} \quad (2)$$

Step 3: Derivative of the denominator

The denominator is

$$\text{denominator}(z) = \sum_{k=1}^K \text{relu}(z_k - \max(z) + b),$$

so

$$\frac{\partial \text{denominator}}{\partial z_j} = \sum_{k=1}^K \frac{\partial}{\partial z_j} \text{relu}(z_k - \max(z) + b).$$

For each k , by the same reasoning as above,

$$\frac{\partial}{\partial z_j} \text{relu}(z_k - \max(z) + b) = \begin{cases} \delta_{kj} - \delta_{tj}, & \text{if } z_k - \max(z) + b > 0, \\ 0, & \text{if } z_k - \max(z) + b < 0. \end{cases} \quad (3)$$

Thus we can write the derivative of the denominator more compactly as

$$\frac{\partial \text{denominator}}{\partial z_j} = \sum_{\substack{k=1 \\ z_k - \max(z) + b > 0}}^K (\delta_{kj} - \delta_{tj}). \quad (4)$$

Step 4: Final expression for the Jacobian

Substituting (2) and (4) into the quotient rule (1), we distinguish two cases.

Case 1: $z_i - \max(z) + b < 0$. Then $\text{numerator}_i(z) = 0$ and $\frac{\partial \text{numerator}_i}{\partial z_j} = 0$, so from (1)

$$\frac{\partial \text{relumax}_b(z)_i}{\partial z_j} = 0.$$

Case 2: $z_i - \max(z) + b > 0$. Then $\frac{\partial \text{numerator}_i}{\partial z_j} = \delta_{ij} - \delta_{tj}$ and

$$\frac{\partial \text{denominator}}{\partial z_j} = \sum_{\substack{k=1 \\ z_k - \max(z) + b > 0}}^K (\delta_{kj} - \delta_{tj}).$$

Inserting these into (1) and using

$$\text{relumax}_b(z)_i = \frac{\text{numerator}_i(z)}{\text{denominator}(z)},$$

we can divide numerator and denominator in (1) by $\text{denominator}(z)$ to obtain

$$\frac{\partial \text{relumax}_b(z)_i}{\partial z_j} = \frac{(\delta_{ij} - \delta_{tj}) - \text{relumax}_b(z)_i \sum_{\substack{k=1 \\ z_k - \max(z) + b > 0}}^K (\delta_{kj} - \delta_{tj})}{\sum_{k=1}^K \text{relu}(z_k - \max(z) + b)}.$$

Putting both cases together, we can write the Jacobian entries as

$$\frac{\partial \text{relumax}_b(z)_i}{\partial z_j} = \begin{cases} 0, & \text{if } z_i - \max(z) + b < 0, \\ \frac{(\delta_{ij} - \delta_{tj}) - \text{relumax}_b(z)_i \sum_{k=1}^K (\delta_{kj} - \delta_{tj})}{\sum_{k=1}^K \text{relu}(z_k - \max(z) + b)}, & \text{if } z_i - \max(z) + b > 0, \end{cases}$$

where t is the (assumed unique) index such that $z_t = \max(z)$.

Question 3.5: Cross-entropy loss with sparsemax or relumax

The cross-entropy loss $L(z) = -\log(\text{output}_i)$ is very unsuitable for use with Sparsemax or Relumax because, unlike Softmax (which outputs strictly positive probabilities due to the exponential function), these alternative functions are designed to produce sparse distributions, meaning they can assign an exact probability of zero to certain classes. If the target class i is assigned $\text{sparsemax}(z)_i = 0$ or $\text{relumax}(z)_i = 0$, the loss explodes to $L(z) = -\log(0) = +\infty$, rendering the entire training sample useless, creating immediate numerical instability, and preventing effective gradient-based learning. As stated by Martins and Astudillo, this issue, where "some labels may have exactly probability zero, so any model that assigns zero probability to a gold label would zero out the probability of the entire training sample," is "highly undesirable" [4].

The original loss function, after applying the logarithm quotient rule and ignoring the constant normalization term $\log(1 + K\epsilon)$, simplifies to:

$$L(z) = -\log(\text{relumax}_b(z)_i + \epsilon)$$

We find the gradient $\nabla_z L(z)$ by applying the Chain Rule directly.

The Chain Rule states: $\nabla_z L(z) = \frac{dL}{d(\text{inner function})} \cdot \nabla_z(\text{inner function})$.

Let the inner function be $f(z) = \text{relumax}_b(z)_i + \epsilon$.

1. **Derivative of the Outer Function:** The outer function is $g(f) = -\log(f)$.

$$\frac{dL}{df} = \frac{d}{df}[-\log(f)] = -\frac{1}{f} = -\frac{1}{\text{relumax}_b(z)_i + \epsilon}$$

2. **Gradient of the Inner Function:**

$$\nabla_z(\text{relumax}_b(z)_i + \epsilon) = \nabla_z(\text{relumax}_b(z)_i) + \nabla_z(\epsilon)$$

Since ϵ is constant, $\nabla_z(\epsilon) = \mathbf{0}$.

$$\nabla_z(\text{relumax}_b(z)_i + \epsilon) = \nabla_z(\text{relumax}_b(z)_i)$$

3. Combine Terms:

$$\nabla_z L(z) = \left[-\frac{1}{\text{relumax}_b(z)_i + \epsilon} \right] \cdot \nabla_z (\text{relumax}_b(z)_i)$$

The final simplified gradient is:

$$\boxed{\nabla_z L(z) = -\frac{\nabla_z (\text{relumax}_b(z)_i)}{\text{relumax}_b(z)_i + \epsilon}}$$

The jacobian of relumax was calculated in the previous question.

References

- [1] Navneet Dalal and Bill Triggs. Histograms of oriented gradients for human detection. In *2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'05)*, volume 1, pages 886–893. IEEE, 2005.
- [2] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.
- [3] Kurt Hornik, Maxwell Stinchcombe, and Halbert White. Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5):359–366, 1989.
- [4] André F. T. Martins and Ramón Fernandez Astudillo. From softmax to sparsemax: A sparse model of attention and multi-label classification, 2016.
- [5] Guido Montúfar, Razvan Pascanu, Kyunghyun Cho, and Yoshua Bengio. On the number of linear regions of deep neural networks. *arXiv preprint arXiv:1402.1869*, 2014.